

拓扑 AC | CSP-S&NOIP 联测

TopoAC Algorithm Contest

第三轮 第二场

题目名称	花圃围栏	帆影定位	归并切换	旅行计划
题目类型	传统型	传统型	传统型	传统型
目录	frame	sail	merge	fuelpath
可执行文件名	frame	sail	merge	fuelpath
输入文件名	frame.in	sail.in	merge.in	fuelpath.in
输出文件名	frame.out	sail.out	merge.out	fuelpath.out
每个测试点时限	2.0 秒	2.0 秒	1.0 秒	2.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	25	25	25	25
子任务数目	2	4	3	6
测试点是否等分	否	否	否	否

提交源程序文件名

对于 C++ 语言	frame.cpp	sail.cpp	merge.cpp	fuelpath.cpp
-----------	-----------	----------	-----------	--------------

编译选项

对于 C++ 语言	-O2 -std=c++14 -Wall
-----------	----------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）和其他可能造成不公平的方法。
7. 请注意所有题目均使用子任务计分，只有通过属于该子任务的全部数据后，才能获得该子任务的分数。

花圃围栏 (frame)

【题目描述】

一座花圃被划分为 n 行 m 列的网格，第 i 行第 j 列格子的观赏值为整数 $a_{i,j}$ ，观赏值可以为负数。

你需要选择一条与网格边平行、恰好一格宽的矩形围栏。用

$top, bottom, left, right$ 表示围栏所在矩形的上、下、左、右边界，它们必须满足：

$$1 \leq top < bottom \leq n, \quad 1 \leq left < right \leq m$$

围栏覆盖的格子集合是以下两类格子的并集：

- 第 top 行和第 $bottom$ 行中，第 $left$ 列至第 $right$ 列的所有格子；
- 第 $left$ 列和第 $right$ 列中，行号严格大于 top 且严格小于 $bottom$ 的所有格子。

因此，围栏的高和宽都至少为 2 个格子，四个角上的格子各计算一次，围栏内部未被覆盖的格子不计入。

你必须选择一条合法围栏。请最大化围栏覆盖格子的观赏值之和。即使所有格子的观赏值均为负数，也不能不选，此时答案可以为负数。

【输入格式】

从 `frame.in` 读入数据。

第一行输入两个整数 n, m ，分别表示网格的行数和列数。

接下来 n 行，每行输入 m 个整数。其中第 i 行的第 j 个整数表示 $a_{i,j}$ 。

【输出格式】

输出到 `frame.out` 文件。

仅输出一个整数，表示一条合法围栏所能取得的最大观赏值之和。

【样例 1 输入】

```
1 5 4
2 3 -2 -2 -4
3 0 4 2 1
4 5 -1 0 -2
5 -6 -1 1 -1
6 1 3 -1 0
```

【样例 1 输出】

```
1 10
```

【样例 2 输入】

```
1 3 3
2 5 5 5
3 5 -100 5
4 5 5 5
```

【样例 2 输出】

```
1 40
```

【样例 3 输入】

```
1 2 4
2 -8 -1 -9 -2
3 -7 -3 -6 -4
```

【样例 3 输出】

```
1 -19
```

【样例 4】

输入与输出见 sample 目录下的 *frame/frame4.in* 和 *frame/frame4.ans*。

【样例解释】

样例 1

可以选择第 2 行至第 3 行、第 1 列至第 3 列的矩形。它的高为 2，因此这 6 个格子都在围栏上，观赏值之和为 10。

样例 2

选择整个 3×3 矩形时，围栏覆盖外围的 8 个格子，观赏值之和为 40。中心格子不属于围栏，四个角上的格子各计算一次。

样例 3

例如，选择第 1 行至第 2 行、第 1 列至第 2 列的矩形，围栏观赏值之和为 -19。虽然所有观赏值均为负数，仍必须选择一条合法围栏。

【数据范围与子任务】

对于所有测试数据，保证：

$2 \leq n, m \leq 10^5, \quad n \times m \leq 2 \times 10^5, \quad |a_{i,j}| \leq 100$

本题采用捆绑测试。 只有通过一个子任务内的全部测试，才能获得该子任务的分数。

子任务	分数	附加约束
1	50	$n \leq 100$ 且 $m \leq 100$
2	50	无额外限制

帆影定位 (sail)

【题目描述】

海岸观测图是一张 $n \times m$ 的字符网格。第 i 行第 j 列为 $.$ 时表示该位置可用，为 $*$ 时表示该位置被遮挡。

一个帆影由四个整数 (r, h, c_1, c_2) 确定，其中 $h \geq 2$ 且 $c_1 < c_2$ 。它占据 h 行：在网格第 $r + t$ 行 ($0 \leq t < h$)，占据的列区间是

$$[c_1 - (h - 1 - t), c_2 + (h - 1 - t)].$$

也就是说，帆影的最下面一行占据 $[c_1, c_2]$ ，每向上一行就向左右各扩展一格。

四元组 (r, h, c_1, c_2) 合法，当且仅当：

1. 它的所有行、列都在网格内，即 $1 \leq r$ 、 $r + h - 1 \leq n$ 、 $1 \leq c_1 - (h - 1)$ 、 $c_2 + (h - 1) \leq m$ ；
2. 上述所有被帆影占据的格子都是 $.$ 。

两个四元组只要有任意一个参数不同，就视为两种不同方案。请计算合法帆影的总数。答案不取模。

【输入格式】

从 *sail.in* 读入数据。

第一行两个整数 n, m 。

接下来 n 行，每行一个长度为 m 、只包含 $.$ 和 $*$ 的字符串。

【输出格式】

输出到 *sail.out* 文件。

输出一个整数，表示合法帆影的数量。

【样例 1 输入】

```
1 3 5
2 .....
3 *.***
4 **.***
```

【样例 1 输出】

```
1 3
```

【样例 2 输入】

```
1 4 5
2 .....
3 .....*
4 .....
5 .....
```

【样例 2 输出】

```
1 7
```

【样例 3 输入】

```
1 10 10
2 .....
3 ..*.....*..
4 .....*...
5 .....
6 .....
7 ...*.....
8 .....
9 .....
10 *.*.*.*.
11 .....
```

【样例 3 输出】

```
1 104
```

【样例 4】

输入与输出见 sample 目录下的 *sail/sail4.in* 和 *sail/sail4.ans*。

【数据范围与子任务】

对所有数据， $1 \leq n, m \leq 2000$ 。
采用子任务捆绑计分。

子任务	分值	附加限制
1	20	$n, m \leq 50$
2	20	$m \leq 3$
3	20	网格中没有 *
4	40	无额外限制

归并切换 (merge)

【题目描述】

需要将一个长度为 n 、元素为 $1, 2, \dots, n$ 的排列按升序排序。排序过程固定如下：

1. 若当前连续段长度不超过 k ，使用一次直接排序将该段排好；这一步总能成功，不产生归并切换限制。
2. 否则，将区间 $[l, r]$ 固定拆成 $[l, \lfloor (l+r)/2 \rfloor]$ 和 $[\lfloor (l+r)/2 \rfloor + 1, r]$ ，递归排好两段，再合并两个有序段。

合并时，每次取出两个段当前最小的元素。若本次取出的元素与上一次来自不同的段，就产生一次“切换”；第一次取元素不算切换。一次合并中的切换次数若大于 k ，排序失败。

求 $1, 2, \dots, n$ 的所有排列中，能使整棵递归排序过程的每一次合并都成功的排列数。答案对 998244353 取模。

【输入格式】

从 *merge.in* 读入数据。

一行两个整数 n, k 。

【输出格式】

输出到 *merge.out* 文件。

输出合法排列数模 998244353 的结果。

【样例 1 输入】

1 4 1

【样例 1 输出】

1 8

【样例 2 输入】

1 4 2

【样例 2 输出】

1 16

【样例 3 输入】

1 10 4

【样例 3 输出】

1 1296000

【样例 4 输入】

1 100 12

【样例 4 输出】

1 702581085

【样例 5】

输入与输出见 sample 目录下的 *merge/merge5.in* 和 *merge/merge5.ans*。

【数据范围与子任务】

$1 \leq n, k \leq 500$ 。采用子任务捆绑计分。

子任务	分值	附加限制
1	25	$n \leq 10$
2	25	$k \leq 2$
3	50	无附加限制

旅行计划 (fuelpath)

【题目描述】

有 n 座城市通过 $n - 1$ 条双向公路连接，公路网是一棵树。每条公路行驶时恰好消耗 1 升汽油。

城市 i 有一座补给站，补给值为 a_i 。在城市 i 使用补给站后，汽车的油量会变为 a_i 升；这不是在原有油量上增加 a_i 升。

你需要处理 m 次操作：

- C u x：把城市 u 的补给值修改为 x ；
- Q s t：规划一次从城市 s 到城市 t 的旅程。

每次询问都是一次独立的旅程。旅程开始时，汽车位于 s ，油量为 0。汽车必须按顺序沿 s 到 t 的唯一简单路径行驶，不能离开这条路径或回头。行驶一条公路前，油量必须至少为 1 升；行驶后油量减少 1 升。

在一次旅程中，路径上每座城市的补给站至多使用一次，起点 s 的补给站也可以使用。一次询问中的补给选择不会影响后续询问；只有 C 操作会永久修改补给值。到达城市 t 时旅程立即结束，因此无需在 t 补给。

对于每次询问，求到达 t 至少需要使用多少座城市的补给站。

【输入格式】

从 *fuelpath.in* 读入数据。

第一行一个整数 n 。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ 。

接下来 $n - 1$ 行，每行两个整数 u, v ，表示城市 u 与城市 v 之间有一条双向公路。

下一行一个整数 m 。

接下来 m 行，每行是以下两种格式之一：

- Q s t；
- C u x。

【输出格式】

输出到 *fuelpath.out* 文件。

对于每次 Q 操作，输出一行一个整数，表示该次旅程最少需要使用的补给站数量。

【样例 1 输入】

```
1 3
2 2 1 1
3 1 2
4 2 3
5 4
6 Q 1 3
7 Q 3 1
8 C 3 3
9 Q 3 1
```

【样例 1 输出】

```
1 1
2 2
3 1
```

【样例 2 输入】

```
1 5
2 2 2 1 1 1
3 1 2
4 2 3
5 3 4
6 4 5
7 5
8 Q 1 5
9 Q 5 1
10 C 5 4
11 Q 5 1
12 Q 2 5
```

【样例 2 输出】

```
1 3
2 4
3 1
4 2
```

在样例 2 的第一次询问中，经过城市 2 时使用补给站，会把当时的油量直接变为 2 升，而不是在原有油量上增加 2 升；最少需要补给 3 次。

【样例 3】

输入与输出见 sample 目录下的 *fuelpath/fuelpath3.in* 和 *fuelpath/fuelpath3.ans*。

【数据范围】

对于所有数据：

- $1 \leq n, m \leq 10^5$ ；
- $1 \leq a_i, x \leq 20$ ；
- 所有城市编号都在 $[1, n]$ 内；
- 输入的 $n - 1$ 条公路构成一棵树；
- 每次询问满足 $s \neq t$ 。

正式数据按下表分为六个相互独立的捆绑子任务。

子任务	分值	附加条件
1	20	$n, m \leq 5000$
2	20	$n, m \leq 30000$ ，且所有补给值始终不超过 3
3	10	$n, m \leq 70000$ ，且所有补给值始终不超过 5
4	20	树为 $1 - 2 - \dots - n$ 的链，且每次询问均有 $s < t$
5	20	$n, m \leq 70000$ ，且所有补给值始终不超过 10
6	10	无额外限制